

TP Hadoop HDFS Exercice 1 : Analyse et Lancement

Nom :Amanetoullah Cheikh

Mai 2026

Exercice 1 : Analyse et Lancement

1. Analyse du fichier docker-compose.yml

Le fichier `docker-compose.yml` définit un cluster Hadoop composé d'un NameNode et de deux DataNodes.

Dans une configuration classique, lorsqu'on souhaite exposer plusieurs DataNodes sur la machine hôte, il est nécessaire d'utiliser des ports différents. Par exemple, on peut mapper `9865:9864` pour le deuxième DataNode afin d'éviter un conflit avec le premier qui utiliserait `9864:9864`.

Cependant, dans notre cas, aucun port n'est explicitement exposé pour les services `datanode1` et `datanode2`. Les deux conteneurs utilisent uniquement le port interne `9864/tcp`, comme observé avec la commande :

```
docker-compose ps
```

Cela signifie que les DataNodes communiquent uniquement via le réseau interne Docker, sans passer par des ports externes. Par conséquent, il n'y a aucun conflit de ports.

Conclusion : Le mapping `9865:9864` est une solution générale pour éviter les conflits de ports, mais dans notre configuration, il n'est pas nécessaire car les ports ne sont pas exposés.

2. Diagnostic avec docker-compose ps

Après le démarrage du cluster avec la commande :

```
docker-compose up -d
```

nous avons vérifié l'état des conteneurs avec :

```
docker-compose ps
```

Le résultat montre que tous les services sont en état **healthy**.

Le statut **healthy** indique que le conteneur fonctionne correctement et qu'il a passé avec succès les tests de vérification interne (healthcheck). Dans le cas du NameNode, cela signifie qu'il est opérationnel et prêt à gérer les métadonnées du système HDFS.

3. Validation via l'interface Web

L'interface Web du NameNode est accessible à l'adresse suivante :

```
http://localhost:9870
```

En accédant à la section **Datanodes**, nous observons les nœuds actifs du cluster.

Dans notre cas, deux DataNodes sont listés comme étant actifs ("Live") :

- datanode1
- datanode2

Nombre de DataNodes "Live" : 2

Cela confirme que le cluster Hadoop est correctement déployé et que les deux DataNodes communiquent correctement avec le NameNode.

1 Exercice 2 : Ingestion et Consultation des données dans HDFS

1.1 Objectif

L'objectif de cet exercice est de manipuler un dataset réel (NYC Taxi Trip) en utilisant HDFS afin de comprendre :

- la création de répertoires dans HDFS,
- le transfert de données vers HDFS,
- le concept de réplication,
- et la gestion des fichiers distribués.

1.2 Préparation de l'environnement

Avant l'ingestion des données, un cluster Hadoop fonctionnel a été déployé à l'aide de Docker, composé de :

- un NameNode,
- deux DataNodes,
- un système de fichiers HDFS opérationnel.

Le dataset utilisé est un fichier CSV de taille approximative 562 Mo.

1.3 Création du répertoire cible dans HDFS

La première étape consiste à créer un répertoire dans HDFS pour organiser les données :

```
hdfs dfs -mkdir -p /tphdfs/data
```

Analyse :

- La commande `mkdir` permet de créer un répertoire dans HDFS.
- L'option `-p` permet de créer les répertoires parents si nécessaire.
- Le chemin `/tphdfs/data` sera utilisé comme zone de stockage des données du TP.

1.4 Chargement du dataset dans HDFS

Le fichier local est ensuite copié vers HDFS :

```
hdfs dfs -put /tp/data/nyctrip.csv /tphdfs/data/
```

Analyse :

- Le fichier est transféré depuis le système local (Docker container) vers HDFS.
- HDFS découpe automatiquement le fichier en blocs.
- Chaque bloc est distribué sur les DataNodes.

1.5 Vérification du fichier dans HDFS

Pour vérifier l'existence du fichier :

```
hdfs dfs -ls /tphdfs/data/
```

Résultat obtenu :

```
-rw-r--r--  3 root supergroup 562386202 nyctrip.csv
```

Analyse :

- Le fichier est bien présent dans HDFS.
- La taille du fichier est d'environ 562 Mo.
- Le chiffre **3** indique le facteur de réplication.

1.6 Vérification du facteur de réplication

Le facteur de réplication est vérifié avec la commande :

```
hdfs dfs -stat "%r" /tphdfs/data/nyctrip.csv
```

Résultat :

3

Analyse :

- Par défaut, Hadoop applique un facteur de réplication égal à 3.
- Cela signifie que chaque bloc du fichier est stocké sur trois DataNodes différents.
- Ce mécanisme assure la tolérance aux pannes et la haute disponibilité.

1.7 Modification du facteur de réplication

Le facteur de réplication est ensuite modifié à 2 :

```
hdfs dfs -setrep -w 2 /tphdfs/data/nyctrip.csv
```

Analyse de la commande :

- L'option **-setrep 2** modifie le nombre de copies du fichier.
- L'option **-w** (wait) force Hadoop à attendre la fin de la réplication.

Importance de **-w** :

- Sans **-w**, la commande retourne immédiatement sans garantir la mise à jour complète.
- Avec **-w**, Hadoop attend que tous les blocs soient correctement répliqués.

1.8 Conclusion

Cet exercice a permis de :

- manipuler HDFS pour stocker un dataset réel,
- comprendre le fonctionnement du stockage distribué,
- observer le découpage en blocs,
- et analyser la réplication des données.

Ainsi, Hadoop assure la fiabilité des données grâce à la réplication et la distribution sur plusieurs nœuds.

2 Exercice 3 : Analyse des Blocs (Le cœur du Big Data)

2.1 Objectif

Cet exercice vise à analyser le fonctionnement interne de HDFS, notamment :

- le découpage des fichiers en blocs,
- la distribution des blocs sur les DataNodes,
- la tolérance aux pannes,
- et le rôle de la réplication.

2.2 Analyse du système de fichiers (FSCK)

La commande suivante est utilisée pour analyser le fichier dans HDFS :

```
hdfs fsck /tphdfs/data/nyctrip.csv -files -blocks -locations
```

Résultat global : Le système indique un état **HEALTHY**, ce qui signifie que le fichier est intact et correctement stocké dans HDFS.

2.3 Découpage en blocs

Le fichier de taille approximative 562 Mo est découpé en plusieurs blocs.

Analyse :

- Taille du fichier : 562 Mo
- Taille d'un bloc HDFS : 128 Mo

Calcul :

$$562 \div 128 \approx 4.4 \Rightarrow 5\text{blocs}$$

Conclusion : Le fichier est donc découpé en **5 blocs**.

Remarque : HDFS ne stocke jamais un fichier entier dans un seul bloc afin de permettre le traitement distribué.

2.4 Pourquoi le fichier n'est pas stocké en un seul bloc ?

HDFS découpe les fichiers pour les raisons suivantes :

- permettre le traitement parallèle des données,
- améliorer les performances du système,
- répartir la charge sur plusieurs machines,
- assurer la scalabilité du système Big Data,
- garantir la tolérance aux pannes.

2.5 Localisation des blocs

Les blocs du fichier sont répartis sur les DataNodes du cluster avec un facteur de réplication.

Analyse : Chaque bloc est stocké sur plusieurs nœuds afin d’assurer la disponibilité des données.

2.6 Tolérance aux pannes

Pour tester la tolérance aux pannes, un DataNode est arrêté :

```
docker stop datanode1
```

Le fichier est ensuite lu avec :

```
hdfs dfs -cat /tphdfs/data/nyctrip.csv | head
```

Résultat : Le fichier reste accessible.

Explication : Grâce à la réplication des blocs (facteur 2 ou 3), chaque bloc existe sur plusieurs DataNodes. Ainsi, même si un nœud tombe en panne, les autres copies permettent de continuer à accéder aux données.

2.7 Remarque importante

HDFS est conçu pour fonctionner dans des environnements distribués où les pannes sont fréquentes. La réplication garantit donc :

- la disponibilité des données,
- la robustesse du système,
- et la continuité du service.

2.8 Conclusion

Cet exercice démontre que HDFS :

- découpe les fichiers en blocs,
- distribue les blocs sur plusieurs machines,
- réplique les données pour la sécurité,
- et assure la tolérance aux pannes.

Ainsi, Hadoop garantit un stockage fiable et distribué adapté aux systèmes Big Data.

3 Exercice 4 : Gestion des flux dans HDFS (appendToFile)

3.1 Objectif

Cet exercice a pour objectif de manipuler les flux de données dans HDFS en ajoutant des informations à un fichier existant sans le ré-uploader entièrement, à l’aide de la commande `appendToFile`.

3.2 Création du fichier de log local

Une première étape consiste à créer un fichier local contenant une nouvelle ligne de log :

```
echo "[2026-05-07 07:30] INFO: Nouveau taxi ajouté" > log.txt
```

Ce fichier est créé dans le conteneur NameNode et n'est pas encore stocké dans HDFS.

3.3 Ajout du flux dans HDFS

La commande suivante est utilisée pour ajouter le contenu du fichier local vers HDFS :

```
hdfs dfs -appendToFile log.txt /tphdfs/data/personnes.txt
```

Observation : Dans un premier temps, une erreur liée au lease recovery et au pipeline de DataNode est apparue, empêchant l'écriture.

```
Failed to replace a bad datanode on the existing pipeline
```

Après stabilisation du cluster (redémarrage et synchronisation des DataNodes), la commande a pu être exécutée avec succès.

3.4 Vérification des fichiers HDFS

La commande suivante permet de vérifier les fichiers présents dans HDFS :

```
hdfs dfs -ls /tphdfs/data/
```

Résultat obtenu :

```
nyctrip.csv
personnes
personnes.txt
```

On observe la présence de deux fichiers similaires : `personnes` et `personnes.txt`.

3.5 Analyse des résultats

Deux lectures différentes ont été effectuées :

— `personnes.txt` contient le contenu initial :

```
Amanetoullah
```

— `personnes` contient la ligne ajoutée via `append` :

```
[2026-05-07 07:30] INFO: Nouveau taxi ajouté
```

3.6 Conclusion

L'utilisation de la commande `appendToFile` permet d'ajouter des données dans un fichier HDFS sans ré-uploader l'ensemble du fichier. Cependant, une mauvaise gestion du chemin peut entraîner la création d'un nouveau fichier au lieu de modifier celui attendu.

Cet exercice met en évidence deux points importants :

- HDFS fonctionne en mode distribué et nécessite des DataNodes opérationnels pour les écritures.
- La précision des chemins est essentielle, car un chemin différent entraîne la création d'un fichier distinct.

Ainsi, HDFS garantit la robustesse et la cohérence des données, mais impose une rigueur dans la manipulation des fichiers.

4 Exercice 5 : Clôture et maintenance du cluster

4.1 1. Arrêt du cluster

Pour arrêter l'ensemble du cluster Hadoop déployé avec Docker, la commande suivante est utilisée :

```
docker-compose down
```

Cette commande arrête tous les conteneurs (NameNode et DataNodes) ainsi que le réseau Docker associé, mais ne supprime pas automatiquement les volumes de données.

4.2 2. Réflexion sur la persistance des données

Lors du redémarrage du cluster sans l'option `-v`, les données HDFS ne sont pas perdues.

Explication : Les fichiers HDFS sont stockés dans des volumes Docker persistants. Ainsi, même après l'arrêt des conteneurs, les données restent disponibles lors du redémarrage du cluster.

Cependant, si l'option `-v` est utilisée :

```
docker-compose down -v
```

les volumes sont supprimés, entraînant la perte totale des données HDFS.

4.3 3. Commande critique et danger

Une commande particulièrement dangereuse en environnement de production est :

```
docker-compose down -v
```

ou encore :

```
docker system prune -a --volumes
```

Analyse du danger : Ces commandes suppriment l'ensemble des conteneurs, images et volumes Docker. Dans un environnement Big Data comme Hadoop, cela entraîne la suppression irréversible des données stockées dans HDFS.

4.4 Conclusion

La gestion du cycle de vie d'un cluster Hadoop doit être effectuée avec prudence. L'utilisation de Docker permet de simplifier le déploiement et la maintenance, mais une mauvaise utilisation des commandes de nettoyage peut entraîner une perte totale des données distribuées.

5 Tableau récapitulatif des concepts Hadoop

Concept	Définition / Utilité constatée pendant le TP
NameNode	Le NameNode est le nœud maître de HDFS. Il gère les métadonnées du système de fichiers (structure des répertoires, localisation des blocs, permissions). Dans le TP, il permet de visualiser et contrôler les fichiers via les commandes <code>hdfs dfs</code> .
DataNode	Les DataNodes sont les nœuds esclaves qui stockent physiquement les blocs de données. Dans le TP, ils assurent la lecture, l'écriture et la réplication des fichiers comme <code>nyctrip.csv</code> . Leur disponibilité est essentielle pour les opérations d'écriture.
Block Size	Le Block Size correspond à la taille maximale d'un bloc dans HDFS (souvent 128 MB par défaut). Dans le TP, le fichier <code>nyctrip.csv</code> (562 MB) a été automatiquement découpé en plusieurs blocs pour être distribué sur les DataNodes.
Replication Factor	Le facteur de réplication définit le nombre de copies de chaque bloc dans le cluster. Dans le TP, la réplication a été observée (valeur 2 ou 3 selon configuration), permettant la tolérance aux pannes en cas de perte d'un DataNode.
FSCK	La commande <code>fsck</code> (File System Check) permet de vérifier l'état de santé du système HDFS. Dans le TP, elle a été utilisée pour analyser le fichier <code>nyctrip.csv</code> , vérifier les blocs, leur répartition et confirmer l'intégrité des données.